

Improving Accuracy and Efficiency of Graph Embedding Training with Fine-Grained Parameter Management

Lihan Hu
University of Iowa
lihan-hu@uiowa.edu

Peng Jiang
University of Iowa
peng-jiang@uiowa.edu

Abstract—Efficient and accurate graph embedding learning is crucial for various real-world applications. However, the large-scale nature of graph embeddings poses significant challenges, particularly in managing the massive amount of embedding parameters across CPU and GPU memories. This paper presents a fine-grained parameter management technique that significantly improves the accuracy and efficiency of graph embedding learning. Our approach leverages parameter duplication and a novel precaching strategy, which minimizes the overhead of data movement between CPU and GPU while preventing stale data usage during training. We introduce an analytical model to estimate the access frequency of embeddings, allowing for the optimal placement of embedding data between CPU and GPU. Using a zero-copy data access mechanism, our system effectively reduces training time while maintaining high accuracy. Experimental results on multiple large-scale knowledge graphs demonstrate that our approach achieves substantial performance gains compared to existing methods with improved Mean Reciprocal Rank (MRR).

Index Terms—knowledge graph embedding, data movement

I. INTRODUCTION

Graph embedding (GE) is the process of representing graph nodes and edges as vectors to encode the discrete graph structures in Euclidean space. The learned embeddings (also called embedding vectors) can be used for various machine learning tasks on graphs, such as information extraction [1], recommender systems [2], question answering [3], and link prediction [4]. Despite their wide adoption in real-world applications, training GE models efficiently while maintaining high accuracy is challenging due to their large size and irregular data access.

First, because the GE model maintains an embedding vector for each node in the graph, the model parameters are large. For example, the Freebase Knowledge Graph [5], which stores the general facts extracted from Wikipedia with 86M entities and 14k relations, has over 80 GB of embedding vectors with a vector length of 500. Since GPUs have limited memory, it is often necessary to store the embeddings on CPU for GE training. This significantly slows down the training process as data must be copied back and forth between CPU and GPU in every training iteration. The problem has been confirmed in previous work [6], which shows that an early system adopting

This work was supported by NSF award CNS-2310423.

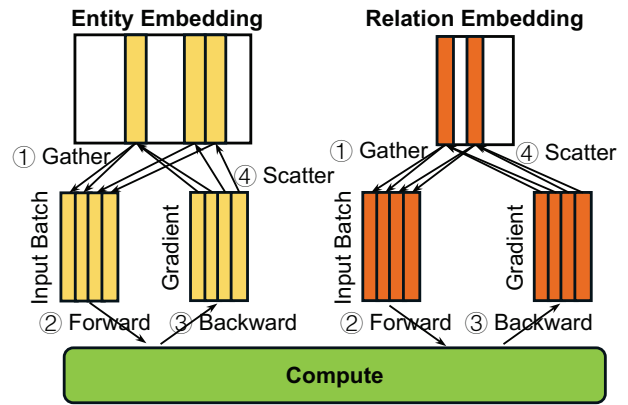


Fig. 1: Data access pattern of training a graph embedding model.

this naive design achieves extremely low GPU utilization due to the large data movement overhead.

The second challenge in training GE models is due to the irregular and random access of model parameters throughout the training process, as shown in Fig. 1. There are two types of embedding vectors in a GE model. The *entity* embeddings are trained to represent connectivity between nodes in the graph, and the *relation* embeddings are trained to represent the characteristics of different relationships. In each iteration, the algorithm samples a small number of edges. A batch of embedding vectors are gathered based on the sampled edges for computing the loss in the forward pass, and the computed gradients are scattered back to the corresponding embedding vectors at the end of the backward pass. Since the edges are randomly sampled, the gathering and scattering of model parameters have poor data locality, exacerbating the data movement overhead issue. Moreover, the irregular data access can also lead to convergence issues in parallel training, as multiple processors may update the same embedding vectors simultaneously (i.e., write-conflicts).

There are two well-known techniques that can be used to reduce data movement overhead: *data prefetching* and *data precaching*. Marius [6] takes the data prefetching approach. It samples edges ahead of time and pre-loads the embedding

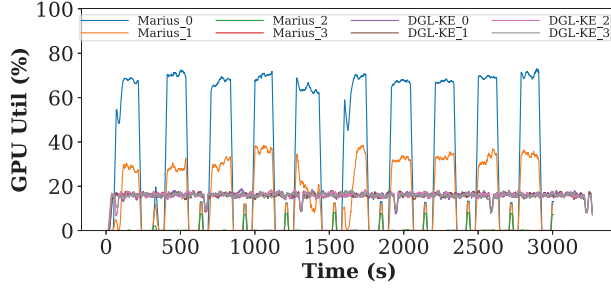


Fig. 2: Scalability issue with data prefetching.

vectors into a pipeline one or multiple iterations before the forward computation uses them. This approach hides the data movement overhead between CPU and GPU; however, it introduces a data staleness issue – the embeddings of prefetched entities or relations might be updated before the fetched data is used. While Marius tries to mitigate this issue by limiting the pipeline length, the data staleness still leads to a noticeable accuracy loss.

Besides the data staleness issue, data prefetching also has poor scalability across multiple GPUs. Marius was originally designed for a single GPU [6]. Its latest distribution on GitHub supports multiple GPUs. However, in multi-GPU training, the CPU becomes the bottleneck as it struggles to fill up the pipelines for different GPUs simultaneously. As shown in Fig. 2, the first two GPUs running Marius (blue and orange lines) show much higher utilization compared to DGL-KE due to data prefetching, while the latter two GPUs suffer from extremely low utilization.

It is also challenging to apply data precaching to GE training. Marius [6] caches all the relation embeddings on GPU because they are relatively small and are more frequently accessed compared to entity embeddings. However, it is hard for Marius to scale out to multiple GPUs with such data precaching because different GPUs may update the same relation embeddings. In fact, its multi-GPU version disables data precaching and stores all the embeddings on CPU. DGL-KE [7] caches relation embeddings on multiple GPUs by partitioning the graph edges based on their relation types. To avoid conflicting updates, it distributes the edges of different relation types to different GPUs and restricts each GPU to only sample edges from its local partition. Such local data sampling usually leads to slower convergence of the training algorithm compared to global sampling [8], [9]. Our experiments also validate this problem (see Sec. V).

To address the limitations of existing GE learning systems, we propose a fine-grained parameter *duplication-and-precaching* technique. Our key idea is to use different updating and caching strategies for different embedding vectors based on their access frequency. To resolve the write-conflict issue with global edge sampling, we duplicate the frequently accessed embedding vectors for each GPU so that they can be updated locally. The duplicate embeddings are synchronized among the GPUs with an *all-reduce* operation every certain

TABLE I: GE terminology and notation: N is the number of nodes, M is the number of relations, Dim is the embedding length, and B is the batch size.

Notation	Description
$E_{emb} \in \mathbb{R}^{N \times Dim}$	embeddings of all entities
$R_{emb} \in \mathbb{R}^{M \times Dim}$	embeddings of all relations
$Score \in [0..N]^B$	scores of positive edges
$Grad_E \in \mathbb{R}^{N \times Dim}$	gradients of E_{emb}
$Grad_R \in \mathbb{R}^{M \times Dim}$	gradients of R_{emb}

number of iterations to ensure convergence. For embedding vectors that are infrequently sampled, since write conflicts rarely happen, all GPUs share one single copy of the data on CPU. Unlike DGL-KE and Marius, which simply put the entire relation embeddings on GPUs, our system caches both entity and relation embedding vectors. Depending on the access frequency distribution and GPU memory size, all or part of the duplicated embeddings may be cached on GPUs.

To summarize, this paper makes the following contributions:

- We identify the issue of the accuracy-efficiency trade-off in previous GE learning systems.
- We propose a fine-grained parameter duplication-and-precaching technique that improves the data access efficiency of GE training with better accuracy.
- We propose an analytical model to estimate the access frequency of embedding vectors to better arrange them on GPU and CPU memory.
- Our experiments with three GE models and three input graphs show that our system achieves 0.5% to 4.4% higher MRR while reducing the training time by 1.27x to 2.1x compared to DGL-KE.

II. BACKGROUND

This section provides background on graph embedding learning and the CPU-GPU data transfer mechanism to facilitate our discussion in the paper.

A. Graph Embedding Learning

A knowledge graph is a directed, labeled, multi-edge graph where the nodes represent entities and edges represent relations in a certain domain. Graph embedding (GE) encompasses a set of techniques that convert knowledge graphs into continuous, low-dimensional vector representations. GE has gained significant attention in recent years due to its ability to capture semantic relationships between entities, thereby enhancing the accuracy of knowledge graph completion and related tasks. Several notable KGE models have been proposed, including TransE [10], ComplEx [11], and DistMult [12]. These models define a score function on the triplets and optimize the function to maximize the scores of triplets that exist in the knowledge graphs while minimizing the scores of triplets that do not exist.

GEs are trained based on the open world assumption, which states that a knowledge graph contains only true facts, and non-observed facts can be either false or missing [13]. A GE model is typically trained in a mini-batch approach. In each training iteration, a mini-batch of triplets that exist

Algorithm 1: KGE Training Procedure

```
1 for  $i$  in range ( $num\_batches$ ) do
2    $Edges_i = \text{getBatchEdges}(i)$ ;
3    $Eemb_p, Eemb_n = \text{lookupEEmb}(Eemb, Edge_i)$ ;
4    $Remb_p, Remb_n = \text{lookupREmb}(Remb, Edge_i)$ ;
5    $Score_p = f(Eemb_p, Remb_p)$ ;
6    $Score_n = f(Eemb_n, Remb_n)$ ;
7    $Grad_E, Grad_R = \text{computeGrads}(Score_p, Score_n)$ ;
8    $\text{updateEmbedding}(Grad_E, Grad_R)$ ;
9 end
```

in the knowledge graph is sampled, and for each triplet, a large number of negative facts (i.e., non-existing edges) are constructed. A common method to sample negative edges is to replace (corrupt) either the head or tail entity of a positive edge with a set of random entities [14]. All the negative edges corrupted from the same positive edge have the same relation type.

Algorithm 1 illustrates the commonly used knowledge graph embedding training procedure, based on the notation in Table I. Before training begins, the entity and relation embeddings are randomly initialized. At the start of the training process, a mini-batch of edges is sampled, consisting of triplets (head node, tail node, and the relation types between these two nodes). Next, embeddings are look up based on the sampled edges to obtain a mini-batch of embedding vectors. Subsequently, scores are computed using the batch of embeddings according to the graph embedding models for both positive and negative edges. The gradients are then calculated from the positive and negative scores using automatic differentiation. Finally, embeddings are updated through a gradient descent routine aimed at minimizing the training loss.

B. CPU-GPU Data Transfer Mechanism

There are three methods for data transfer between hosts and GPUs: Direct Memory Access (DMA) APIs [15], Unified Virtual Memory (UVM) [16], and zero-copy access [17].

The first method involves Direct Memory Access (DMA) APIs. CUDA provides both synchronous and asynchronous APIs for data transfer between the host and devices. The two most commonly used APIs are *cudaMemcpy()* and *cudaMemcpyAsync()*. While it is efficient for transferring a single large data block, it is sub-optimal for transferring small-sized data due to the request setup latency arising from interactions between the program and operating system.

As the second way, Unified Virtual Memory (UVM), also well-known as Page migration, introduced by NVIDIA for user-friendly programming. Memory regions managed by UVM can be accessed by both the GPU and the CPU. When a processor requires access to a page not present in its local memory, the page will be migrated from a remote location. To allocate UVM-backed memory regions, programmers need only call *cudaMallocManaged()* with the desired size. However, the performance of UVM is constrained by irregular

access patterns, which result in a high page miss rate. This causes an excessive number of page faults, thereby stalling execution and leading to I/O read amplification.

The third method is zero-copy access, also known as direct access. In zero-copy access, the GPU issues cacheline-sized memory requests directly through the external interconnect without the need for explicit data copying or page migration, as occurs in DMA or UVM methods. The source memory can be accessed from host memory, peer PCIe devices, or other GPUs connected via NVLink. While zero-copy is efficient for accessing fine-grained data, it requires GPU cores to actively generate memory requests.

C. Shortcomings of Existing KGE Systems

Many systems have been developed to accelerate KGE training on different platforms [6], [7], [18]. For edge sampling, looking up embedding from CPU memory and transfer to GPU memory brings massive data communication, which significantly slows down the KGE training. To achieve higher efficiency during KGE training, some KGE systems attempted to store the embedding data in GPU global memory. However, for large-scale graphs with tens of millions of entity nodes, such a vast amount of embedding exceeds the GPU memory limitation and cannot be directly stored in GPU global memory. Consequently, the data caching pattern and the overhead caused by frequent data transfers between the CPU and GPU need to be carefully considered.

It is straightforward to store embedding in CPU memory and cache them to GPU in single-GPU training, but for the update stage in multi-GPU training, if some GPUs update the embedding that stored on CPU memory simultaneously, the gradient information may miss, which is also known as write-conflict. Consider a naive implementation of multi-GPU GE training shown in Fig. 3a. The entity and relation embeddings are shared among all GPUs. Each GPU can read and write any embedding vectors. When multiple GPUs sample edges of the same entities or relations, write conflicts occur on the shared embedding vectors, leading to slower convergence of the training algorithm.

DGL-KE mitigates the write-conflict issue by partitioning the relation embeddings on GPUs and only sharing entity embedding on the CPU, as shown in Fig. 3b. It assumes that most write conflicts occur on the relation embeddings and avoids the conflicts by restricting each GPU to sample from edges of relations in its local partition. However, such local sampling also slows down the convergence of the training algorithm [8], [9], [19].

Also, in DGL-KE, each iteration must wait for the previous iteration to complete its training before the new iteration starts, leading to low GPU utilization. Marius [6] addresses the issue of low GPU utilization by designing a pipeline architecture to hide the overhead of data loading and data writing within the score function computation. However, the pipeline design introduces the staleness problem and cannot be scaled to multi-GPU settings. When fetching a batch of embeddings in an iteration, the previous iteration has not yet completed

its update. This means that the batches of embeddings used during training are out-of-date.

III. FINE-GRAINED PARAMETER DUPLICATION AND PRECACHING

To mitigate write conflicts without disabling global sampling, we choose to duplicate embedding vectors instead of partitioning them. The duplicated embeddings are updated locally and synchronized across processors with all-reduce operations. This approach is known as *data-parallel* training and is widely used for training DNNs in a distributed setting [20]. However, data-parallel training cannot be practically applied to GE training because the duplicated embeddings take up a lot of memory space, and the all-reduce operations will incur a large overhead. Since only a small fraction of embeddings are updated in each training iteration, our idea is to duplicate only the most frequently accessed vectors and store them on GPUs for synchronization, as shown in Fig. 3c.

This section explains how our system identifies the most frequently accessed embeddings and how it duplicates and caches them on GPUs.

A. Estimating Access Frequency of Embedding Vectors

In multi-GPU GE training, mini-batches are uniformly sampled by each GPU from all the edges in the graph. As real-world graphs usually have skewed degree distribution, some entity nodes appear on more edges than others, and thus are more likely to be sampled. The relation types of edges also follow a skewed distribution. This skewness leads to data locality and is the basis of our design.

To estimate the access frequency of embedding vectors, we propose a method to calculate the probabilities of entity nodes and relation types being sampled. To simplify the calculation, we use the binomial distribution to approximate this probability.

$$P_{entity} = 1 - (1 - \frac{c_1}{E})^B * (1 - \frac{c_2}{E})^B \quad (1)$$

$$P_{relation} = 1 - (1 - \frac{c_3}{E})^B \quad (2)$$

Here, P_{entity} is the probability of an entity node being sampled in a batch. $P_{relation}$ is the probability of a relation type being sampled in a batch. B is the batch size. E is the count of edges in the graph. c_1, c_2 and c_3 are the count of head nodes, tail nodes and relation types respectively.

The probability of an entity node being sampled at least once can be reduced to calculating the probability of that entity node not being sampled. The probability of a node not being sampled consists of two parts: the probability of not being sampled as a head entity on any edge, and the probability of not being sampled as a tail entity on any edge. The probability of not being sampled as a head entity can be calculated as $1 - c_1/E$ where c_1/E is the probability of sampling that entity as a head. Since we train graph embedding in mini-batches and sample B different edges in each batch, the resulting probability is $(1 - c_1/E)^B$. Similarly, we can

determine the probability of not being sampled as a tail entity. The probability of an entity node being sampled at least once can be represented as Formula 1. Similarly, we can calculate the probability of relation types being sampled at least once with Formula 2.

Fig. 4 shows our estimated access frequency and real frequency after normalization () on Freebase dataset. In Fig. 4a, we show the frequency at which relations are accessed. At sorted relation ID 1000, our estimated frequency is 0.1728, while the real frequency is 0.1901. At sorted relation ID 2000, the estimated frequency is 0.0315 and the real frequency is 0.0353. This indicates that when edges are sampled in the graph, the frequently accessed relations are indeed sampled more often across iterations. We observe a similar phenomenon among the frequency of entity nodes, as shown in Fig. 4b. Overall, our estimated access frequency closely aligns with the real sampling frequency.

Note that we only show the frequency of the first 20,000 entity nodes because the frequency of the subsequent nodes is so low that it can be disregarded. The detail information of Freebase dataset can be found in Table IV. For a given batch, only 15% of relation types have a high probability of being sampled. Interestingly, we find that a very small fraction (no more than 0.001%) of entity nodes have a significant probability of being sampled, which means there are also potential write-conflict for these entity embedding during KGE multi-GPU training. This indirectly suggests that we cannot solely focus on addressing write-conflict issues in relation embeddings; some entity embeddings also have severe write conflicts.

B. Resolving Write Conflicts with Parameter Duplication

When edges are sampled independently on multiple GPUs, they may access the same entity or relation embedding vectors. Intuitively, the entity nodes and relation types that are accessed frequently have a higher possibility of being sampled simultaneously by multiple GPUs in an iteration. In contrast, entities and relations with much lower access frequency are less likely to have write conflicts. Therefore, addressing the write conflicts associated with higher-frequency entities and relations can improve the quality of graph embeddings and accelerate convergence.

Based on our observation, we propose a fine-grained parameter duplication strategy as shown in Fig. 5. We split the embeddings into three parts: the frequently accessed embedding, entity embedding, and relation embedding. We duplicate the frequently accessed embeddings for each GPU. Within the duplicated embeddings, we further partition them into the most frequently accessed ones (shown as MF in the figure) and the less frequent ones.

In our implementation, we sort the ID of all entity nodes and relation types based on their access frequencies and select the top $(p_1 + p_2) \times (L_e + L_r)$ entities/relations as the frequent ones. Here, p_1 is the threshold for the most frequently accessed embeddings, p_2 is the threshold for the less frequently accessed embeddings, and they are hyperparameters that can be adjusted

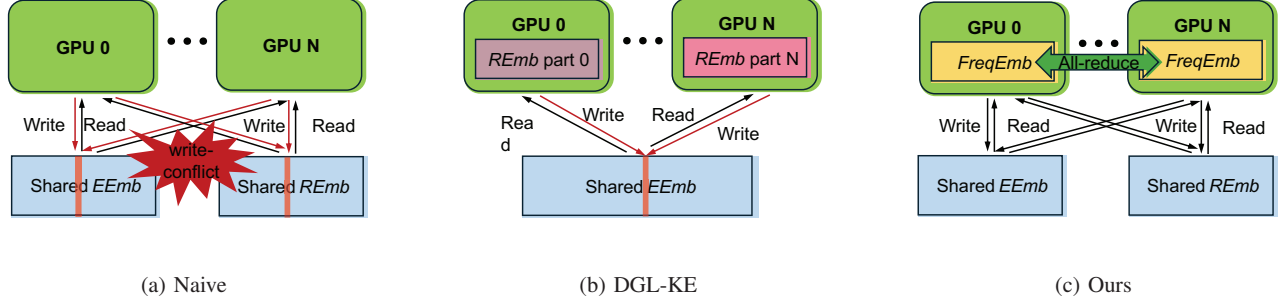


Fig. 3: Comparison of a naive implementation, DGL-KE, and our proposed method for multi-GPU GE training. The red marks indicate where the write-conflict may occur.

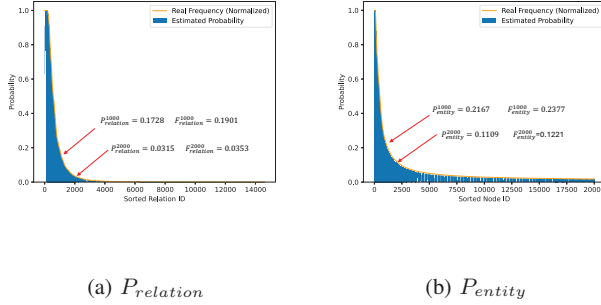


Fig. 4: Comparison of estimated access frequency and real sample frequency on Freebase dataset.

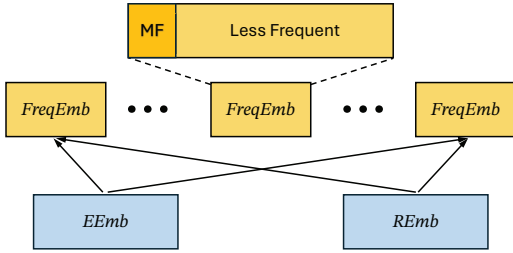


Fig. 5: Embedding duplication: MF is the most frequently accessed embedding.

to determine how many embeddings to be duplicated. L_e and L_r represent the number of entity nodes and relation types, respectively.

Our method ensures that each processor has its own independent set of embedding data for updating. To maintain the data consistency through different processors, we perform an all-reduce operation after updating the parameters.

C. Improving Data Access Efficiency with Precaching

For large graphs, storing the whole entity embedding in the GPU is impossible because of the limited GPU global memory size. Although previous work shows that directly accessing CPU memory can achieve good performance for graph learning [21], it is still less efficient than accessing data directly on GPU. Thus, we propose a precaching strategy to further improve data access efficiency. The idea is to place the most frequently accessed embedding on the GPUs before

```

1 def data_read(Emb, Lemb, Idx, GIdx, off){
2   bid = blockIdx.x * blockDim.y + threadIdx.y;
3   if (GIdx[Idx[bid] + off] == -1) {
4     /* read data from global */
5     for (j=threadIdx.x; j<dim; ++j){
6       LoadEmb[bid][j] = Emb[bid][j];
7     }
8   } else {
9     /* read data from local */
10    for (j=threadIdx.x; j<dim; ++j){
11      LoadEmb[bid][j] = Lemb[GIdx[bid] + off][j];
12    }
13  }
14 }

```

Listing 1: GPU kernel for zero-copy data reading.

training begins. Note that depending on the graph size and GPU memory limit, it is possible that the GPU cannot hold all the duplicate embeddings. For the remaining duplicate embeddings, we store them on CPU memory and set them as pinned memory for GPU direct access. This data caching on GPU not only improves the access efficiency of the most frequent embeddings but also reduces their synchronization overhead by utilizing the fast interconnect among GPUs (i.e., NVLink).

IV. IMPLEMENTATIONS OF DATA ACCESS AND SYNCHRONIZATION

This section describes the implementation details of our fine-grained parameter management strategy.

A. GPU Kernels for Zero-copy Data Access

For the infrequently accessed embeddings stored on CPU, we map their CPU addresses to the GPU address space (known as pinned memory). This allows the GPU to directly access the data from the CPU. More specifically, we call `cudaHostRegister()` to page-lock the embeddings and set the flag parameter to `cudaHostRegisterPortable` so that all CUDA contexts are able to zero-copy access the pinned memory, not just the CUDA device that performed allocation.

To check whether a specific embedding vector is stored on the CPU or GPU, we build an index named $GIdx$, which has a length of the total number of entity nodes plus the number of relation types. The $GIdx$ is constructed by analyzing access frequency before training begins. Each element in $GIdx$ is initially set to -1, indicating that the data should be accessed from CPU memory. Specifically, when an embedding is duplicated

```

1 def data_write(Eemb, Remb, Lemb,
2 Temb, SVemb, Idx, GIdx, off, n_proc){
3   bid = blockIdx.x * blockDim.y + threadIdx.y;
4   if (GIdx[Idx[bid]] == -1) {
5     if (Idx[bid] < off) {
6       /* write data to entity embedding */
7       for (j=threadIdx.x; j<dim; ++j){
8         Eemb[Idx[bid]][j] = Temb[bid][j] + SVemb[bid][j];
9       }
10    }
11    else{
12      /* write data to relation embedding */
13      for (j=threadIdx.x; j<dim; ++j){
14        Remb[Idx[bid]-off][j] = Temb[bid][j] + SVemb[bid][j];
15      }
16    }
17    else{
18      /* write data to local buffer */
19      for (j=threadIdx.x; j<dim; ++j){
20        Lemb[GIdx[Idx[bid]]][j] = Temb[bid][j] * n_proc + SVemb[bid][j];
21      }
22    }
23  }
24 }

```

Listing 2: GPU kernel for zero-copy data writing.

and cached on the GPU, the corresponding element in $GIdx$ is updated to reflect its index in the duplicated embedding array.

Listing 1 explains our zero-copy data read kernel for knowledge graph training. Here, Emb is the entity or relation embedding stored in CPU pinned memory. $Lemb$ denotes the frequently accessed embedding, which is stored in GPU global memory. Idx represents the merged entity and relation indices. b_{id} indicates the index of the data node in the given batch. off is a flag used to mark whether the accessed embedding is relation embedding or entity embedding. If $GIdx$ equals to -1, it signifies that the embedding is on CPU pinned memory. Then the zero-copy data reading kernel loads the corresponding embedding from Emb to $LoadEmb$. Otherwise, the data is stored on the GPU. In this case, the kernel retrieves the data from the GPU local buffer ($Lemb$) and loads it into $LoadEmb$.

Similar to Listing 1, Listing 2 explains how to efficiently write back embeddings during the update phase. $Temb$ denotes the gradient obtained through SGD optimizer. $SVemb$ represents the batch of pre-stored embedding that will be updated. n_{proc} is the number of processors, in this case is the number of GPUs. If $GIdx$ equals to -1, the data write kernel will branch to update the either $Eemb$ or $Remb$ by comparing off . Then it assigns the sums from $Temb$ and $SVemb$ to the embedding. Otherwise, if the embedding is located on the GPU, our kernel updates $Lemb$ by combining values from $Temb$ and $SVemb$. Note that the $Temb$ is multiplied by n_{proc} times, this is because all-reduce averages embeddings across different GPUs. If only $Temb$ is retained during the update, the embeddings after all-reducing would be diluted, which could severely impact the quality of the trained embeddings and lead to a decrease in accuracy. These two kernels together enable efficient and accurate data access during the training process.

B. In-place Parameter Update

Since zero-copy follows a sparse data access pattern, repeated access by multiple GPUs to data in pinned memory can cause PCIe congestion, severely affecting performance. To avoid performance being limited by PCIe bandwidth, we

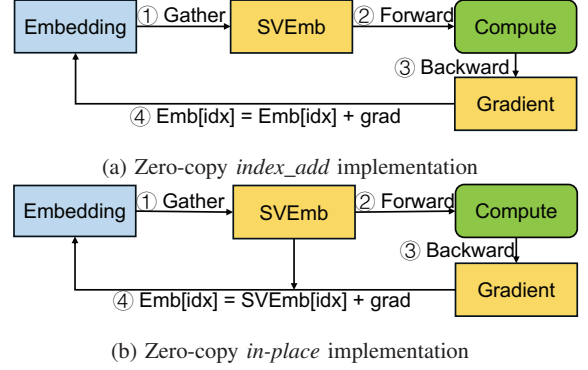


Fig. 6: Comparison of different update implementations.

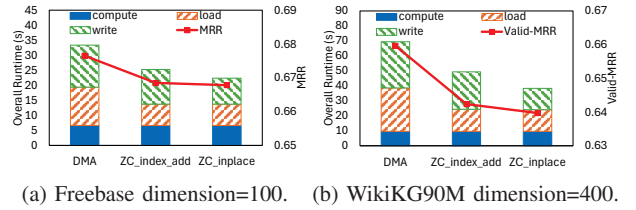


Fig. 7: Comparison of zero-copy implementation on ComplEx model. ZC_index_add represents the *index_add* kernel for zero-copy technique. $ZC_inplace$ represents the *in-place* kernel for zero-copy technique.

did not implement an *index_add* kernel for zero-copy data writing. Instead, we use the pre-stored embedding during computation for updating (as shown in Fig. 6b). Compared with the *index_add* kernel implementation shown in Fig. 6a, we do not need to look up embedding from pinned memory when updating.

Fig. 7 shows the comparison of zero-copy implementation for ComplEx model on Freebase [5] and WikiKG90M [22] dataset. Compared with DMA data access, zero-copy *index_add* implementation achieves significant speedup in data load but does not success in data write. The MRR also drops a lot due to the stale embedding are loaded. Our zero-copy *in-place* implementation significantly speeds up data writing compared to *index_add* implementation. To verify whether using pre-stored embeddings during the update phase introduces more staleness issues, we compare the MRR of zero-copy *index_add* implementation with zero-copy *in-place* implementation in Fig. 7. The results indicates that the MRRs obtained by both implementations are nearly equivalent. This outcome is attributed to the fact that frequently accessed embeddings are stored in the local buffer and synchronized using all-reduce operation, ensuring that the local buffers is not subject to staleness. As for the embeddings stored in CPU pinned memory, the likelihood of being accessed simultaneously by multiple GPUs is extremely low due to the low frequency of entity nodes, as we discussed in Section III-A. Therefore, our zero-copy *in-place* write implementation

does not suffer from staleness issues.

C. GPU Buffer Synchronization

When locally updating the embeddings during multi-GPU training, different GPUs sample different triplets. As different GPUs have a large possibility to access the same embedding simultaneously, we need to synchronize the local buffers across GPUs to ensure parameter consistency. However, it is intuitive that the overhead of all-reduce is related to the buffer size, the larger the local buffer size is, the more significant the overhead of the all-reduce operation becomes. In extreme cases, the overhead of synchronization can exceed the time of all other processes. This introduces the challenge of finding the tradeoff between embedding accuracy and data access overhead in the training process. In our implementation, to efficiently synchronize local GPU buffers across GPUs, we use the all-reduce operation that is implemented by PyTorch with NCCL backend.

Algorithm 2: Segmented Parameter Synchronization

Input: $Lemb_{cpu}, Lemb_{gpu}, p_1, L_e, L_r, t_1, t_2$

```

1  $F_{most} \leftarrow p_1 \times (L_e + L_r);$ 
2  $F_{less} \leftarrow t_1 \times F_{most};$ 
3 while  $iter \in 1, 2, \dots, B$  do
4   General Graph Embedding Training;
5    $all\_reduce(Lemb_{gpu}[: F_{most}]);$ 
6   if  $iter \bmod t_1 == 0$  then
7      $all\_reduce(Lemb_{gpu}[F_{most} : F_{less}]);$ 
8   else if  $iter \bmod t_2 == 0$  then
9      $all\_reduce(Lemb_{gpu}[F_{less} :]);$ 
10     $all\_reduce(Lemb_{cpu});$ 
11 end
```

To optimize parameter settings for higher accuracy with minimal overhead, we propose a parameter synchronization strategy. As shown in Algorithm 2, we have the duplication embedding $Lemb$ and the threshold p_1 from our parameter duplication method, where the first $p_1 \times (L_e + L_r)$ embeddings in the local buffer $Lemb$ are defined as the most frequent accessed embedding. To maintain model accuracy within an acceptable range, we partition the less frequently accessed embedding into two parts and schedule different update periods t_1 and t_2 . For data in the local buffer that is within ten times the size of the most frequently accessed embeddings, we consider the probability of them being accessed simultaneously by multiple GPUs to still be high. Therefore, we choose to perform an all-reduce operation every t_1 iterations for these embeddings. For the remaining embeddings including the less frequently accessed embedding on CPU ($Lemb_{cpu}$) and GPU ($Lemb_{gpu}$), we opt to execute an all-reduce operation every t_2 iterations.

We explore the impact of different buffer sizes on the overall runtime under two different synchronization strategies. Table II displays the breakdown runtime and the percentage of all-reduce time cost for 1000 iterations of training the TransE_L2

Strategy	Load	Write	All-reduce(Percentage)	Total
A-FPM(e^{-4})	4.87	6.89	0.79 (6.29%)	12.55
A-FPM(e^{-3})	4.69	6.79	6.7 (36.85%)	18.18
A-FPM(e^{-2})	4.01	5.54	57.76 (85.81%)	67.31
S-FPM(e^{-3})	4.52	6.89	1.44 (11.2%)	12.85
S-FPM(e^{-2})	3.95	5.5	1.97 (17.25%)	11.42

TABLE II: Breakdown runtime (in seconds) for 1000 iterations with a dimension of 50 in TransE_L2 model on Freebase dataset using four RTX 3090 GPUs.

model on the Freebase dataset using four RTX 3090 GPUs, with a dimension size of 50 and a batch size of 10,000. A-FPM represents synchronizing all the local buffers at every iteration, and S-FPM uses our segmented parameter synchronization strategy. In the A-FPM strategy, the time consumption of the all-reduce operation is almost linearly correlated with the buffer size. When the buffer size threshold $p_1 + p_2$ is set to e^{-4} , since the local buffer is small enough, the overhead of the all-reduce operation is nearly 5% to 10% of the total overhead. Once the buffer size threshold $p_1 + p_2$ is set to e^{-3} , the overhead is approximately 0.3 to 0.4 times that of total overhead. However, when we set $p_1 + p_2$ to e^{-2} , the overhead reaches approximately 0.7 to 0.9 times that of total overhead, which is unacceptable for the latency of the graph embedding training system. Consequently, when buffer size is too large, performing the all-reduce operation for synchronization on all the buffers in every iteration is not a feasible option.

In our segmented parameter synchronization strategy, we set p_1 to e^{-4} , which means the first e^{-4} of the local buffer is regarded as the most frequently accessed embedding, while the buffers beyond that threshold are classified as less frequently accessed embedding. For the data in the local buffer from F_{most} (e^{-4}) to F_{less} (e^{-3}), we synchronize them every t_1 iterations. Then for the last part of the data (e^{-3} to e^{-2} , if applicable) in the local buffer and the buffer that is stored in CPU pinned memory, we synchronize them every t_2 iterations. To simplify the problem, we set t_1 to 10 and t_2 to 100. By this way, the overhead of the all-reduce operation for larger buffer decrease a lot. Comparing the breakdown time of A-FPM and S-FPM in Table II, our segmented parameter synchronization strategy reduced the overhead for all-reduce operations by 4.6 to 29.3 times.

V. EVALUATION

A. Experimental Setup

Datasets: We conducted our experiments on three large-scale knowledge graphs: YAGO25M, Freebase86M, and WikiKG90M, as listed in Table IV. These three graphs are commonly used for evaluating KGE systems [6], [7], [18], [23]. Freebase86M is derived from the Freebased Knowledge Graph [5]. WikiKG90M originates from the Open Graph Benchmark Large-Scale Challenge (OGB-LSC) [22]. YAGO 4 [23] is derived from the YAGO knowledge base, which is based on Wikidata, the largest public general-purpose knowledge base. We construct a subset of YAGO 4 named YAGO25M for evaluation. The test set of WikiKG90M

TABLE III: Score functions of three KGE models: $\|\cdot\|_p$ represents p -norm.

Models	Score Function
TransE [10]	$\ Emb[h] - Emb[t] + Remb[r]\ _{1/2}$
DistMult [12]	$h^T diag(r)t$
ComplEx [11]	$Real(h^T diag(r)t)$

TABLE IV: Knowledge graph datasets.

Name	$\ V\ $	$\ E\ $	$\ R\ $	Epoches
YAGO25M [23]	25M	63.7M	132	10
Freebase86M [5]	86M	338M	14.8k	10
WikiKG90M [22]	87M	504M	1.3k	5

does not provide the true candidate tails, our evaluation on WikiKG90M is performed using the validation dataset. For WikiKG90M experiments, we use Valid-MRR to distinguish it from the test MRR results in other datasets.

Score functions: Table III presents the three score functions utilized in our experiments. We use the distinct embedding for each entity and relation, as normally done in KG embedding models (also called *Shallow*).

Baselines: We compare our system with three baselines: **DGL-KE (ori)** [7]. DGL-KE partitions the graph edges based on the relation types to avoid write-conflict for relation embedding in update phase; **DGL-KE (RP)**. We uniformly partition the graph edges to all the processors for each epoch. This approach may achieve better convergence since each processor explores a wider variety of relation types, leading to more thoroughly trained relation embeddings compared to DGL-KE (ori). However, it requires each processor to access the shared relation embedding; **RP+ZC**. We replace DMA data access with zero-copy data access in DGL-KE (RP). Both entity and relation embeddings are placed in CPU pinned memory, which enables zero-copy technique to reduce the overhead of frequently data transfer from CPU and GPU and bypass intermediate data copying. We also evaluate the performance on Marius. However, since the implementation differences for initialization and sampling [6], the metrics are different in the large-scale graph. We modify Marius to use the same sampling method for a fair comparison, and the result is almost the same as DGL-KE.

Metrics: To clearly demonstrate the performance advantages and accuracy improvements, we validate our experiments using three metrics. **Mean Reciprocal Rank(MRR)** assesses the quality of a model's ranking of correct entities within a list of possible candidates. For each query, the model generates a ranked list of entities, and the reciprocal rank of the correct entity is calculated. The reciprocal rank is defined as the inverse of *rank*, where *rank* is the position of the correct entity in the list; **Hits@k** measures the proportion of times the correct entity or relation appears in the top-*k* ranked positions. It provides an indication of how effectively the model can prioritize relevant results by comparing its ranked predictions with the ground truth, where higher values of Hits@k suggest that the model frequently ranks the correct answers within

the top recommendations. **Overall Runtime** comprises data loading, data writing, all-reduce operations and computation time.

Platform and Settings: Our experiments are conducted on one local workstation with a dual-socket Intel Xeon Gold 6226R CPU (16 Cores, 32 Threads) @2.9GHz with 512 GB RAM and four NVIDIA RTX3090 GPUs @1.4 GHz (82 SMs, 24 GB, 936.2 GB/s bandwidth), each pair of two GPUs connected with NVLink (with NVTAGS NV4) and other GPUs are connected by multiple PCIe bridges. The operating system of our local workstation is Ubuntu 20.04.6 LTS. We also test on a GPU server from Delta cluster with two AMD EPYC 7763 CPUs (64 Cores, 128 Threads) @2.45GHz and 2 TB RAM. The cluster is equipped with eight NVIDIA A100 GPUs @765 MHz (each has 108 SMs, 40 GB, 1.56 TB/s), which are connected all together with NVLink (with NVTAGS NV12). The operating system of Delta cluster is Red Hat Enterprise Linux 8.8. Our CUDA code is compiled on CUDA 11.8 toolkit and NVCC V11.8.89. We collect the overall runtime every 1000 iterations to evaluate the performance comprehensively.

B. Performance Analysis

WikiKG90M Fig. 8 illustrates the results of the ComplEx model on WikiKG90M dataset, using two different dimension sizes and a batch size of 10000, evaluated on four RTX 3090 GPUs. The MRR of **DGL-KE (RP)** is slightly higher than that of **DGL-KE (ori)**, as **DGL-KE (RP)** employs a global sampling method, which enhances generalization capability compared to relation partition used in **DGL-KE (ori)**. However, this improvement incurs higher time overhead because the relation embeddings are stored on the CPU, whereas **DGL-KE (ori)** can access the relation embeddings directly on the GPU, making it more efficient compared to calling DMA APIs from CPU to GPU. Thanks to the adoption of zero-copy technique, the data access time of **RP+ZC** is nearly half of that in **DGL-KE (RP)**. As shown in Fig. 8a, **RP+ZC** does not result in a significant MRR drop at dimensions of 100. However, at dimensions of 400, as shown in Fig. 8b, the staleness issue with zero-copy becomes particularly evident. This indirectly highlights the need for a more flexible parameter management approach to maintain model accuracy.

As shown in Fig. 8b, Our **A-FPM** e^{-4} achieves the MRR and overall runtime slightly higher than that of **RP+ZC**. This is because the local buffer stored on the GPU is not large enough, causing a significant portion of the frequently accessed embeddings to remain unduplicated and not stored on the GPU. This results in increased all-reduce overhead, but also resolves write-conflict issues, leading to an improvement in MRR. When switching to **A-FPM** e^{-3} , with a tenfold increase in buffer size, the MRR improves significantly in many cases. However, this also causes the all-reduce synchronization time to be quite lengthy, as all buffers are synchronized in every iteration. Our segmented parameter synchronization strategy effectively mitigates this issue, reducing the lengthy all-reduce synchronization time. To provide a clearer comparison of the overall runtime, we exclude the runtime of **A-FPM** e^{-2} from

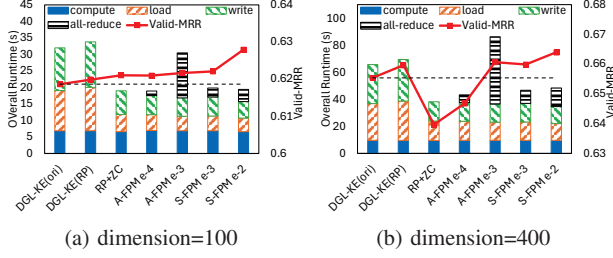


Fig. 8: ComplEx model on WikiKG90M dataset with two dimensions using four RTX 3090 GPUs. The dash line is the MRR of DGL-KE. A-FPM represents all synchronization strategy. S-FPM is our segmented synchronization strategy. e^{-4} , e^{-3} and e^{-2} is the buffer size threshold.

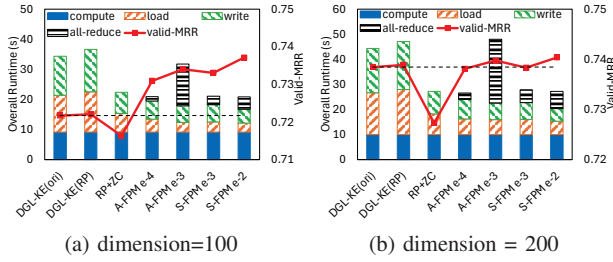


Fig. 9: TransE_I2 model on WikiKG90M dataset with two dimensions using four RTX 3090 GPUs.

the figures, as its all-reduce time far exceeds that of all other overheads (refer to Table II for the breakdown of times). As illustrated in Fig. 8b, **S-FPM** e^{-3} reduces the overall runtime by nearly half compared to **A-FPM** e^{-3} , with less-to-no degradation in MRR. Overall, for ComplEx model, we find that FPM performs better than DGL-KE, achieving a speedup of 1.35x to 1.79x, and an improvement in MRR of 0.002 to 0.02. We also test the TransE_I2 model on WikiKG90M dataset with dimensions of 100 and 200. As shown in Fig. 9, our FPM achieves 1.33x to 1.77x speedup and up to 0.015 MRR improvement compared to DGL-KE.

YAGO25M Fig. 10 demonstrates the results of the TransE_I2 model on YAGO25M dataset using four RTX 3090 GPUs, with dimension sizes of 200 and 400, and a batch size of 10000. Our proposed FPM generally achieves better MRR and shorter overhead compared to **DGL-KE (ori)** and **DGL-KE (RP)**. We find that the all-reduce overhead of **A-FPM** (e^{-3}) is not as substantial on the YAGO dataset compared to the WikiKG90M dataset, this is because the local buffer size is determined by a threshold, and YAGO25M inherently has fewer entities and relations, resulting in a smaller buffer size. The MRR of **ZC + RP** in Fig. 10 does not show a significant decrease compared to **DGL-KE (ori)** and **DGL-KE (RP)** because the YAGO25M dataset contains few edges, and the links in the graph are quite sparse. As a result, the baseline MRR of DGL-KE is not very high in the link prediction task, making the MRR decrease in ZC appear less pronounced. For TransE_I2 model on YAGO25M dataset, our FPM achieves

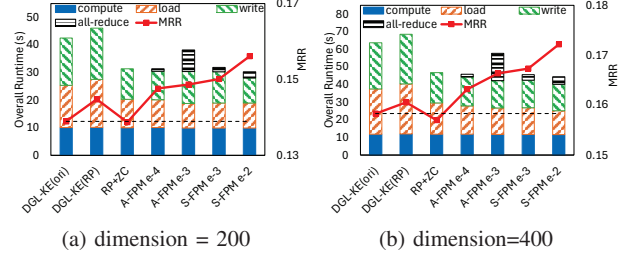


Fig. 10: TransE_I2 model on YAGO25M dataset with two dimensions using four RTX 3090 GPUs.

1.27x to 1.61x speedup and an MRR improvement of 0.007 to 0.044.

Freebase86M Fig. 11 shows the results of the TransE_I2 model on the Freebase86M dataset with dimensions of 100 and 400, using four RTX 3090 GPUs. Although we generally see performance improvements, as the dimension increases, the MRR decline associated with the zero-copy *inplace* implementation also increases. This is because an increase in dimension enhances the expressive capability of graph embeddings. However, in some dense datasets (such as Freebase86M and WikiKG90M), even a small amount of write-conflict can lead to significant data loss. As shown in Fig. 11a, at a dimension of 100, **A-FPM** e^{-3} surpasses DGL-KE in MRR, and there is a noticeable improvement in MRR with **S-FPM** e^{-2} . In Fig. 12b, however, at a dimension of 400, only the **S-FPM** e^{-2} shows a slight increase in MRR. Our FPM method achieves a speedup of 1.29 to 1.49 times on the TransE_I2 model.

To show the scalability of FPM, we evaluate DistMult model on the Freebase86M dataset with two dimensions of 200 and 400 using eight A100 GPUs, as shown in Fig. 12. Since the A100 GPU supports connectivity via NVLink among eight GPUs, the time cost for performing all-reduce operations across these GPUs is relatively low. Therefore, we include the results of **A-FPM** e^{-2} in the figure for comparison. We find that on eight A100 GPUs, the time cost of a single GPU in DGL-KE is longer compared to a single GPU on four RTX 3090 GPUs. This is due to the excessive number of DMA requests causing PCIe congestion between the CPU and GPUs.

Moreover, while our proposed FPM achieves significant speedup and maintains model accuracy, the acceleration obtained during data writing is quite limited compared to the significant performance improvement during data loading. Since this problem does not occur on our local machine with four RTX 3090 GPUs, we believe that the increase in the number of GPUs also leads to a significant rise in the number of zero-copy requests sent to CPU pinned memory, resulting in PCIe congestion and substantially impacting the efficiency of data writing. For training the same number of epochs in a graph embedding model, doubling the number of GPUs effectively halves the number of iterations required. Overall, training a graph embedding model on eight A100 GPUs will still achieve better performance. For DistMult on eight A100 GPUs, our evaluation shows that our proposed FPM achieves

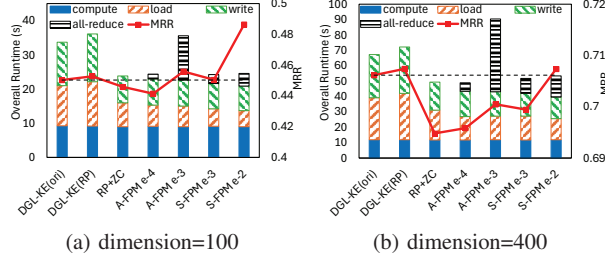


Fig. 11: TransE_L2 model on Freebase86M dataset with two dimensions using four RTX 3090 GPUs.

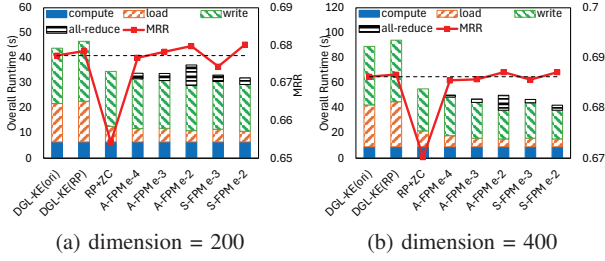


Fig. 12: DistMult model on Freebase86M dataset with two dimensions using eight A100 GPUs.

1.32x to 2.1x speedup compared to DGL-KE.

An interesting observation is that as the dimension size increases, although our FPM approach still achieves speedup, the improvement in MRR appears to become less pronounced. We believe this is because with the increased dimension size and the number of relationships, the embeddings are able to capture more complex relationships and features to enhance their representation, accelerating convergence and reducing the risk of over-fitting. However, the staleness issue introduced by zero-copy persists, leading to the MRR of **A-FPM** e^{-3} being lower than that of **DGL-KE (ori)** and **DGL-KE (RP)** in Fig. 12. Additionally, we find that the WikiKG90M and YAGO25M datasets do not exhibit this problem (as shown in Fig. 8 and Fig. 10). This seems to be because the numbers of relationships in WikiKG90M and YAGO25M are relatively smaller than in Freebase86M, allowing almost all relation embeddings to be stored in the GPU buffer. Consequently, this greatly reduces the issues of write-conflict and staleness for relation embeddings. We aim to address the staleness issue caused by zero-copy and explore potential opportunities for performance improvement in future work.

C. Link Prediction

Since the quality of graph embedding training relies on the dimension length, with larger dimensions indicating stronger generalization capabilities, we compare the MRR and Hits@k for different models at larger dimension sizes. As presented in Table V, the MRR and Hits@k of the DistMult model on the Freebase86M dataset are evaluated with embedding dimensions of 200 and 400, and a batch size of 10000. Although relation partition can cause local convergence issues,

Dim	Strategy	MRR	Hits1	Hits3	Hits10	Hits20
200	DGL-KE(ori)	0.6774	0.6278	0.7013	0.7584	0.7822
	DGL-KE(RP)	0.6812	0.6356	0.7072	0.7602	0.7838
	RP+ZC	0.6683	0.6169	0.6987	0.7579	0.7832
	A-FPM(e^{-4})	0.6831	0.6392	0.7074	0.7602	0.7839
	A-FPM(e^{-3})	0.6834	0.6395	0.7085	0.7606	0.7823
	A-FPM(e^{-2})	0.6851	0.6405	0.71	0.764	0.7874
	S-FPM(e^{-3})	0.6812	0.6378	0.7057	0.7572	0.7797
	S-FPM(e^{-2})	0.6849	0.6414	0.7092	0.7611	0.7839
400	DGL-KE(ori)	0.6883	0.6401	0.7134	0.7664	0.7904
	DGL-KE(RP)	0.6923	0.6479	0.7147	0.7689	0.7915
	RP+ZC	0.6793	0.629	0.7092	0.766	0.7902
	A-FPM(e^{-4})	0.6921	0.65	0.7154	0.7673	0.7907
	A-FPM(e^{-3})	0.6917	0.6494	0.7154	0.7661	0.7881
	A-FPM(e^{-2})	0.6921	0.6498	0.7158	0.7654	0.7876
	S-FPM(e^{-3})	<u>0.6925</u>	<u>0.6503</u>	0.716	0.7675	0.7898
	S-FPM(e^{-2})	0.6927	0.6509	0.7159	<u>0.7685</u>	0.7908

TABLE V: Hits@k for 1000 iterations with a dimension of 200 in DistMult model on Freebase86M dataset. Bold numbers indicate the maximum value in the column. Underlined numbers represent the second highest value in the column.

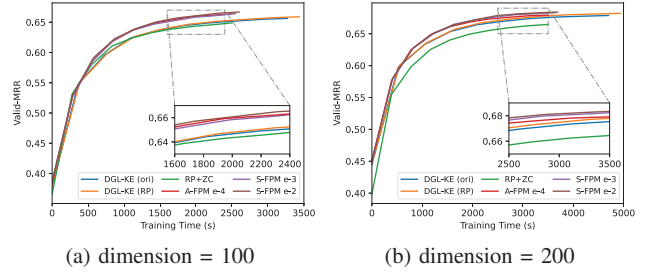


Fig. 13: Convergence comparison for DistMult model on Freebase86M dataset with two dimensions using 4 RTX 3090 GPUs.

in **DGL-KE (RP)**, multiple GPUs need to access the CPU memory simultaneously, which might introduce additional write-conflict issues. Consequently, the MRR and Hits@k of **DGL-KE (RP)** are close to those of **DGL-KE (ori)**.

In our proposed FPM method, frequently accessed embeddings are duplicated and stored in GPU memory, effectively addressing the write-conflict issue. As a result, FPM outperforms **DGL-KE (ori)**, **DGL-KE (RP)**, and **RP + ZC** in terms of MRR and Hits@k when the dimension size is set to 200. While S-FPM performs slightly lower than A-FPM in terms of MRR and Hits@k, the significant improvement in overall runtime compensates for the minimal accuracy loss, which is within a reasonable range. At an embedding dimension of 400, the increased complexity of embeddings introduces some staleness issues due to the zero-copy *inplace* implementation, making the accuracy improvement less pronounced. Nonetheless, compared to **DGL-KE (ori)** and **DGL-KE (RP)**, our FPM method markedly reduces time overhead and greatly enhances GPU utilization in graph embedding training.

D. Convergence Comparison

Fig. 13 shows the convergence comparisons for the DistMult model on the Freebase86M dataset with dimensions of 100 and

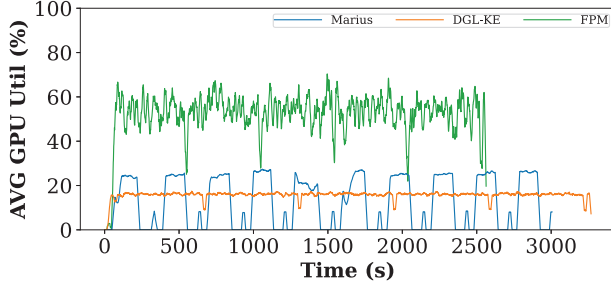


Fig. 14: Average GPU utilization for ComplEx model on Freebase86M dataset with dimension size of 100 using four RTX 3090 GPUs.

200, using four RTX 3090 GPUs. In the initial two epochs, DGL-KE performs better because it neglects a small amount of the gradients for frequently accessed embeddings during updates (write-conflict on entities). These write-conflict can inject a form of “noise” that promotes faster exploration and may yield better initial metrics. However, as training progresses, this conflict-induced randomness undermines stable gradient accumulation. Clearly, starting from the third epoch in Fig. 13, our FPM method surpasses DGL-KE, with this improvement primarily due to our resolution of the write-conflict issues. Moreover, by comparing **DGL-KE (ori)** with **DGL-KE (RP)** in Fig. 13b, it is evident that **DGL-KE(RP)** achieves higher MRR because of global sampling for the graph embedding. In general, our proposed FPM method converges faster and achieves a higher final MRR.

E. GPU Utilization

Although Marius only achieves up to 70% utilization on the first GPU, the utilization of the other three GPUs is extremely low. To demonstrate the advantages of FPM, we evaluate the average GPU utilization over time on DistMult model with dimension of 100 on the Freebase dataset compared to DGL-KE and Marius. As shown in Fig. 14, since Marius requires all embeddings to be stored in CPU memory during multi-GPU training, it does not achieve a significant speedup compared to DGL-KE. Our FPM is the fastest method compared to Marius and DGL-KE, thanks to the use of zero-copy and our data management strategies. In contrast to Marius data-prefetching approach with load imbalance problem, our proposed FPM method achieves an average GPU utilization of over 55% and maintains load balance across multiple GPUs. Compared to DGL-KE with DMA, our method achieves more than three times the GPU utilization of DGL-KE without GPU idle.

VI. RELATED WORK

Previous work has proposed various techniques for graph embedding training. GraphVite [24] focuses on multi-GPU training and keeps all embeddings on CPU memory. It constructs a subgraph and moves the subgraph to GPU global memory. This method reduces the overhead of data movement but introduces staleness of the embedding, resulting in slower

convergence. PyTorch BigGraph [18] is proposed to scale to large graphs and distributed training. It partitions the adjacency matrix of the graph into blocks and assigns blocks to different machines to avoid conflicts. However, the partition swaps between different devices are expensive, causing low GPU utilization, and it does not support GPU training. DGL-KE [7] supports both multi-GPU and distributed training. It partitions the graphs for each GPU based on the relation types and stores the part of relation embedding on the corresponding GPU, avoiding staleness training. But its synchronization is inefficient, GPU will be idle when waiting for the mini-batch communication between CPU and GPU, lead to lower GPU utilization. Marius [6] proposes a pipeline architecture that leverages partition caching and a data reordering scheme to optimize data movement and maximize GPU utilization. It bounds the number of mini-batches in the pipeline to limit staleness, but the accuracy degradation is still noticeable.

Besides graph embedding systems, there are also many systems developed for distributed GNN training. Seungwon et.al [21] proposed a GPU-centric data communication method that enables direct access to host memory for sparse graph data without heavy CPU involvement. DistGNN [25] is a distributed training framework that scales Graph Neural Networks to massive graphs through efficient partition-based scheduling and an optimized communication layer, achieving near-linear speedups across multi-GPU clusters. UGache [26] is designed to accelerate embedding-based deep learning tasks by efficiently managing the movement of large embedding tables between CPU memory and limited GPU memory. The primary difference between GNN training systems and graph embedding training systems lies in how they operate on embeddings. In GNN training systems, the input embeddings are read-only, whereas in graph embedding training systems, embeddings not only need to be read but also updated (written) frequently in each iteration. Comparing with read-only embedding data, data updating during training introduces more challenges and potential optimizations.

VII. CONCLUSION

In this work, we present a fine-grained parameter management strategy that achieve high accuracy and efficiency for large-scale graph embedding training. By introducing parameter duplication and a novel caching strategy, our system effectively reduces the data movement overhead between CPU and GPU while preventing the staleness issue during training. The use of an analytical model to estimate embedding access frequencies allows for optimized memory allocation, further enhancing data access performance. Extensive experiments on multiple knowledge graphs demonstrate that our technique not only achieves training speedup but also maintains or even improves model accuracy. We believe our FPM method can also benefit other areas of machine learning and deep learning, as it largely addresses data transfer and memory management challenges between the CPU and GPU. As for future work, we intend to continue extending FPM to multi-node scenarios and exploring strategies for optimizing CPU and GPU utilization.

REFERENCES

- [1] M. D. Ma, X. Wang, P. Kung, P. J. Brantingham, N. Peng, and W. Wang, "STAR: boosting low-resource information extraction by structure-to-text data generation with large language models," in *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*, M. J. Wooldridge, J. G. Dy, and S. Natarajan, Eds. AAAI Press, 2024, pp. 18 751–18 759.
- [2] F. Elahi, M. Fazlali, H. T. Malazi, and M. Elahi, "Parallel fractional stochastic gradient descent with adaptive learning for recommender systems," *IEEE Trans. Parallel Distributed Syst.*, vol. 35, no. 3, pp. 470–483, 2024.
- [3] J. Ma, P. Wang, D. Kong, Z. Wang, J. Liu, H. Pei, and J. Zhao, "Robust visual question answering: Datasets, methods, and future challenges," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5575–5594, 2024.
- [4] L. Shi, B. Hu, D. Zhao, J. He, Z. Zhang, and J. Zhou, "Structural information enhanced graph representation for link prediction," in *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*, M. J. Wooldridge, J. G. Dy, and S. Natarajan, Eds. AAAI Press, 2024, pp. 14 964–14 972.
- [5] K. D. Bollacker, C. Evans, P. K. Paritosh, T. Sturge, and J. Taylor, "Freebase: a collaboratively created graph database for structuring human knowledge," in *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2008, Vancouver, BC, Canada, June 10-12, 2008*, J. T. Wang, Ed. ACM, 2008, pp. 1247–1250.
- [6] J. Mohoney, R. Waleffe, H. Xu, T. Rekatsinas, and S. Venkataraman, "Marius: Learning massive graph embeddings on a single machine," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI) 21*, 2021.
- [7] D. Zheng, X. Song, C. Ma, Z. Tan, Z. Ye, J. Dong, H. Xiong, Z. Zhang, and G. Karypis, "DGL-KE: training knowledge graph embeddings at scale," in *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, J. X. Huang, Y. Chang, X. Cheng, J. Kamps, V. Murdock, J. Wen, and Y. Liu, Eds. ACM, 2020, pp. 739–748.
- [8] B. Recht and C. Ré, "Toward a noncommutative arithmetic-geometric mean inequality: Conjectures, case-studies, and consequences," in *Conference on Learning Theory. JMLR Workshop and Conference Proceedings*, 2012, pp. 11–1.
- [9] P. Jiang and G. Agrawal, "A linear speedup analysis of distributed deep learning with sparse and quantized communication," *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [10] A. Bordes, N. Usunier, A. García-Durán, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States*, C. J. C. Burges, L. Bottou, Z. Ghahramani, and K. Q. Weinberger, Eds., 2013, pp. 2787–2795.
- [11] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, and G. Bouchard, "Complex embeddings for simple link prediction," in *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, ser. JMLR Workshop and Conference Proceedings, M. Balcan and K. Q. Weinberger, Eds., vol. 48. JMLR.org, 2016, pp. 2071–2080.
- [12] B. Yang, W. Yih, X. He, J. Gao, and L. Deng, "Embedding entities and relations for learning and inference in knowledge bases," 2015.
- [13] L. Drumond, S. Rendle, and L. Schmidt-Thieme, "Predicting RDF triples in incomplete knowledge bases with tensor factorization," in *Proceedings of the ACM Symposium on Applied Computing, SAC 2012, Riva, Trento, Italy, March 26-30, 2012*, S. Ossowski and P. Lecca, Eds. ACM, 2012, pp. 326–331.
- [14] D. Krompaß, S. Baier, and V. Tresp, "Type-constrained representation learning in knowledge graphs," in *The Semantic Web - ISWC 2015 - 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015. Proceedings, Part I*, ser. Lecture Notes in Computer Science, M. Arenas, Ó. Corcho, E. Simperl, M. Strohmaier, M. d'Aquin, K. Srinivas, P. Groth, M. Dumontier, J. Heflin, K. Thirunarayan, and S. Staab, Eds., vol. 9366. Springer, 2015, pp. 640–655.
- [15] A. Thompson and C. Newburn, "Gpudirect storage: A direct path between storage and gpu memory," Tech. Rep., 2019.
- [16] T. N. Allen and R. Ge, "In-depth analyses of unified virtual memory system for GPU accelerated computing," in *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2021, St. Louis, Missouri, USA, November 14-19, 2021*, B. R. de Supinski, M. W. Hall, and T. Gambelin, Eds. ACM, 2021, p. 64.
- [17] D. Ganguly, Z. Zhang, J. Yang, and R. G. Melhem, "Adaptive page migration for irregular data-intensive applications under GPU memory oversubscription," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, New Orleans, LA, USA, May 18-22, 2020. IEEE, 2020, pp. 451–461.
- [18] A. Lerer, L. Wu, J. Shen, T. Lacroix, L. Wehrstedt, A. Bose, and A. Peysakhovich, "Pytorch-biggraph: A large scale graph embedding system," in *Proceedings of Machine Learning and Systems 2019, MLSys 2019, Stanford, CA, USA, March 31 - April 2, 2019*, A. Talwalkar, V. Smith, and M. Zaharia, Eds. mlsys.org, 2019.
- [19] A. Kochsiek and R. Gemulla, "Parallel training of knowledge graph embedding models: A comparison of techniques," *Proc. VLDB Endow.*, vol. 15, no. 3, pp. 633–645, 2021.
- [20] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "Pytorch distributed: Experiences on accelerating data parallel training," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3005–3018, 2020.
- [21] S. Min, K. Wu, S. Huang, M. Hidayetoglu, J. Xiong, E. Ebrahimi, D. Chen, and W. W. Hwu, "Large graph convolutional network training with gpu-oriented data communication architecture," *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 2087–2100, 2021.
- [22] W. Hu, M. Fey, H. Ren, M. Nakata, Y. Dong, and J. Leskovec, "Ogb-lsc: A large-scale challenge for machine learning on graphs," *arXiv preprint arXiv:2103.09430*, 2021.
- [23] T. P. Tanon, G. Weikum, and F. M. Suchanek, "YAGO 4: A reasonable knowledge base," in *The Semantic Web - 17th International Conference, ESWC 2020, Heraklion, Crete, Greece, May 31-June 4, 2020. Proceedings*, ser. Lecture Notes in Computer Science, A. Harth, S. Kirrane, A. N. Ngomo, H. Paulheim, A. Rula, A. L. Gentile, P. Haase, and M. Cochez, Eds., vol. 12123. Springer, 2020, pp. 583–596.
- [24] Z. Zhu, S. Xu, J. Tang, and M. Qu, "Graphvite: A high-performance CPU-GPU hybrid system for node embedding," in *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*, L. Liu, R. W. White, A. Mantrach, F. Silvestri, J. J. McAuley, R. Baeza-Yates, and L. Zia, Eds. ACM, 2019, pp. 2494–2504.
- [25] V. Md, S. Misra, G. Ma, R. Mohanty, E. Georganas, A. Heinecke, D. Kalamkar, N. K. Ahmed, and S. Avancha, "Distgnn: scalable distributed training for large-scale graph neural networks," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021.
- [26] X. Song, Y. Zhang, R. Chen, and H. Chen, "UGACHE: A unified GPU cache for embedding-based deep learning," in *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, J. Flinn, M. I. Seltzer, P. Druschel, A. Kaufmann, and J. Mace, Eds. ACM, 2023, pp. 627–641.